

Restate 101

Build innately resilient distributed applications

A hands-on 101 with the Restate Java SDK

Agenda

1. **What can you build?** — the problem, and the shift
2. **The building blocks** — Services, Virtual Objects, Workflows
3. **How does it work?** — why it's **super** reliable

We'll write real Java and break things on purpose.

Part 1 — What can you build?

The reliability tax

To make a distributed application **reliable** today, you hand-build:

- **Retries** with backoff — and idempotency keys so retries don't double-charge
- **State machines** to track "where did this get to?"
- **Queues / outbox / DLQs** to not lose work across a crash
- **Schedulers / cron** for "do this in 3 days"
- **Sagas** to undo half-finished work
- **Reconciliation jobs** for when it all drifts anyway

This is 80% of the code in a distributed application.
None of it is the business logic.

Durable Execution: the idea

What if the platform remembered **every step your code already did**, so that after **any** crash it just... kept going?

- You write a plain function.
- Each step is **journaled** as it completes.
- On any failure, the function **resumes from where it was** — completed steps are **not** re-run.

That's **durable execution**. Restate is a durable execution engine.

What Restate is

A single, self-contained binary that sits **in front of your services** and makes the plain code you already write durable, exactly-once, and able to hold state.

- Your services stay **stateless to deploy and scale** — Restate manages their state **for** them
- It provides durability, retries, **state**, scheduling, and communication
- **No** Kafka, no separate workflow DB, no Redis, no cron, no Zookeeper to run

Server 1.7.0 · Java SDK 2.8.0 · written in Rust

What you build with it

You want to...	With Restate
Write reliable, durable microservices	durable handlers + exactly - once RPC between them
Money movement / payments	exactly - once steps + compensation
Customer onboarding	Workflows that wait days for humans/3rd parties
Per - account / per - customer state	Virtual Objects — state without a database
"Do X in 3 days", reminders, retries	durable timers — no cron, no scheduler
Event processing	durable handlers off a log/queue
Durable AI agents	tool calls + memory that survive restarts

Not just workflows — durable services

Workflow engines (e.g. **Temporal**) give you **durable workflows** — a single orchestrator function that drives a sequence of steps.

Restate gives you **durable services** — a toolkit for building reliable **systems**:

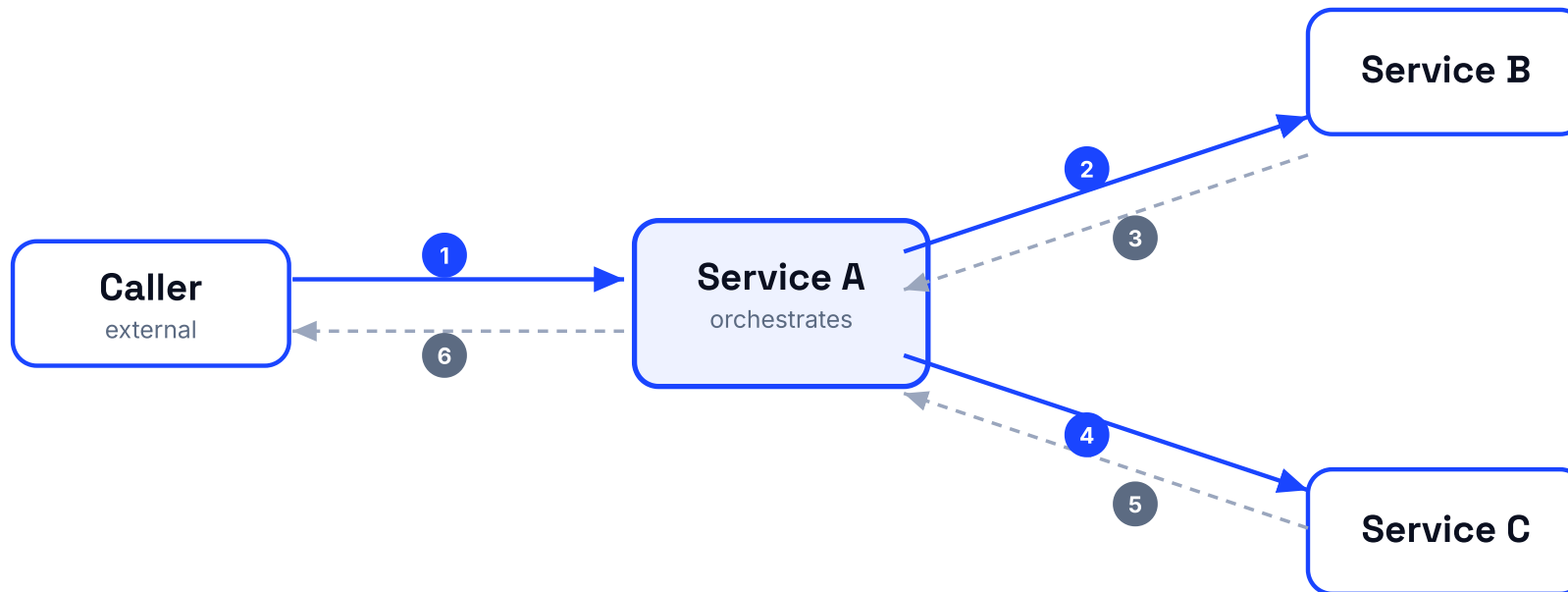
- reliable invocations from the outside world **and between services** (exactly-once RPC)
- every handler internally runs **durable, multi-step execution** — state, timers, retries, promises
- a workflow is just one flavor; so are stateful objects and plain services

	Workflow engine (e.g. Temporal)	Restate
What you build	a workflow	services , stateful objects & workflows
Service-to-service calls	roll your own	durable & exactly-once , built in
To operate	server + DB + search index	one self-contained binary

Part 2 — The building blocks

Request flow — the logical view

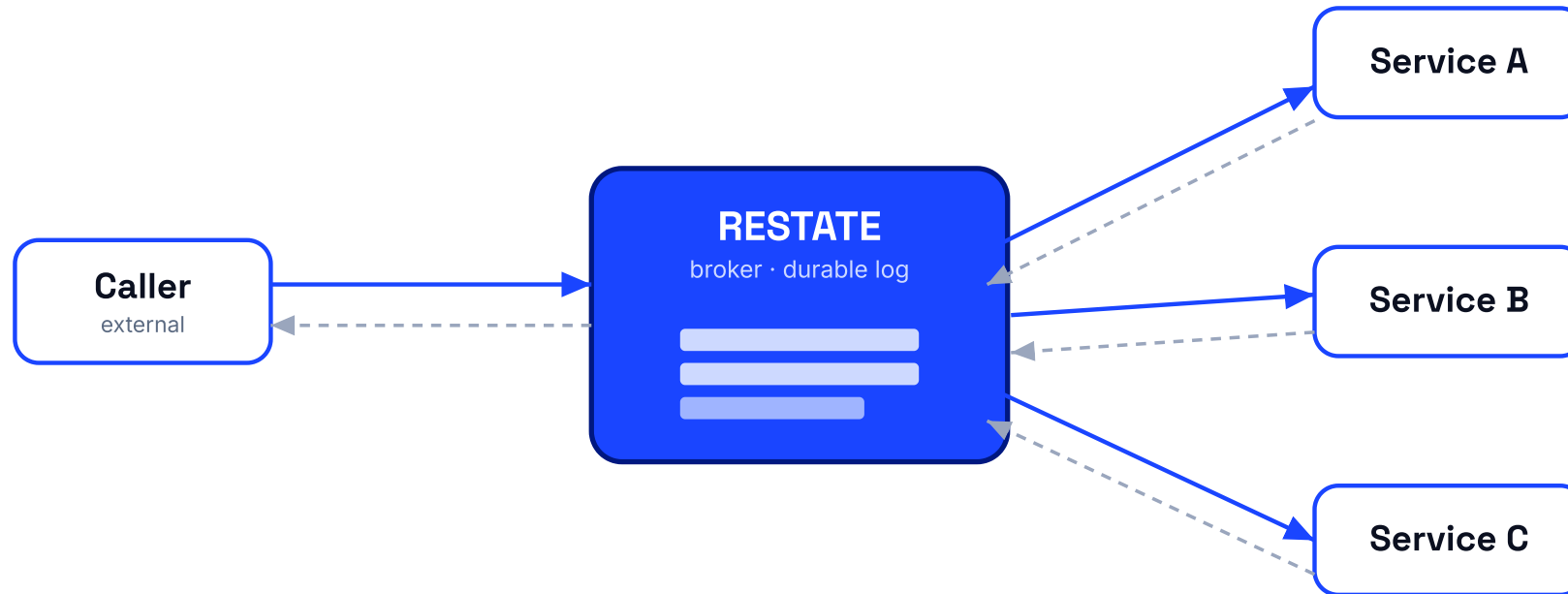
Service A handles a request by calling B, then C, and returns a synthesized result.



1 Caller→A · 2 A→B · 3 B→A · 4 A→C · 5 C→A · 6 A→Caller (synthesized) — just functions calling functions.

Request flow — the physical view

Same logic — but every call is brokered by **Restate**, which journals each hop.



Every hop goes **through Restate and into its log before the target runs** — that recorded journal is what lets any step resume after a crash.

...and here's Service A's code

It just looks like normal function calls — but each one is **durable, exactly-once RPC**.

```
@Service
public class ServiceA {

    @Handler
    public Result handle(Context ctx, Request req) {
        // Looks like a plain method call; under the hood it's brokered
        // and journaled by Restate, and replayed on recovery.
        ResultB b = ServiceBClient.fromContext(ctx).stepB(req.id()).await();
        ResultC c = ServiceCClient.fromContext(ctx).stepC(req.id()).await();

        // Plain Java in between – combine the answers and return.
        return new Result(b.value() + c.value());
    }
}
```

The only differences from code you already write: a **Context ctx** parameter, calls via a generated **...Client.fromContext(ctx)**, and **.await()**.

No queues, no state machine, no retry/idempotency boilerplate.

Three building blocks

	Annotation	Has state?	Concurrency
Service	@Service	no	parallel
Virtual Object	@VirtualObject	yes, per key	one writer per key
Workflow	@Workflow	yes, per id	a run + signals

Same programming model for all three. Pick by what you need.

Block 1 — Service (a durable function)

A set of handlers that run as **durable functions** — each step is journaled, retried, and executed exactly once, even across crashes.

```
@Service
public class SubscriptionService {
    @Handler
    public void add(Context ctx, SubscriptionRequest req) {
        ctx.run("create user", () → UserClient.create(req.userId(), req.creditCard()));

        for (String plan : req.subscriptions()) {
            ctx.run("subscribe: " + plan,
                () → PaymentClient.subscribe(req.userId(), req.creditCard(), plan));
        }
    }
}
```

- Each `ctx.run(...)` is a durable, **retried**, exactly-once step.
- Crash after "create user"? On recovery it skips it and continues at the loop.

Sagas — undo half-finished work

```
List<Runnable> compensations = new ArrayList<>();
try {
  compensations.add(() → ctx.run("undo: delete user", () → UserClient.delete(id)));
  ctx.run("create user", () → UserClient.create(id, card));

  for (String plan : plans) {
    compensations.add(() → ctx.run("undo: cancel " + plan, () → cancel(id, plan)));
    ctx.run("subscribe: " + plan, () → subscribe(id, plan)); // may throw TerminalException
  }
} catch (TerminalException e) {
  for (int i = compensations.size() - 1; i ≥ 0; i--) compensations.get(i).run();
  throw e; // undo, newest first — durably
}
```

A **TerminalException** stops retries. The compensations run — and **they're** durable too, so even the cleanup survives a crash.

Block 2 — Virtual Object (state, no database)

A keyed object that owns **durable state** and serializes writes per key — a per-entity actor with built-in consistency, no database and no locks.

```
@VirtualObject
public class Account {
    private static final StateKey<Long> BALANCE = StateKey.of("balance", Long.class);

    @Handler // exclusive: at most ONE runs per account key at a time
    public long withdraw(ObjectContext ctx, long amountCents) {
        long balance = ctx.get(BALANCE).orElse(0L);
        if (amountCents > balance) throw new RuntimeException("Insufficient funds");
        ctx.set(BALANCE, balance - amountCents);
        return balance - amountCents;
    }

    @Shared // read-only, runs concurrently
    public long balance(SharedObjectContext ctx) { return ctx.get(BALANCE).orElse(0L); }
}
```

State lives in **Restate**, keyed by account. **No locks** — Restate serializes writes per key for you.

Block 3 — Workflow (waits for the world)

A **durable, long-running process** keyed by id that can wait for events, people, or time — and resumes exactly where it left off after any failure.

```
@Workflow
public class PaymentApprovalWorkflow {
    static final DurablePromiseKey<Boolean> DECISION = DurablePromiseKey.of("decision", Boolean.class);

    @Workflow
    public String run(WorkflowContext ctx, PaymentRequest req) {
        ctx.run("notify approver", () → ApprovalClient.requestApproval(ctx.key(), req));
        boolean ok = ctx.promise(DECISION).future().await(Duration.ofDays(3)); // wait up to 3 days
        return ok ? "APPROVED" : "REJECTED";
    }

    @Shared
    public void decide(SharedWorkflowContext ctx, boolean ok) {
        ctx.promiseHandle(DECISION).resolve(ok); // a human (or system) signals
    }
}
```

The workflow **durably blocks** for days. Restart the server mid-wait — it resumes.

The toolbox on `ctx`

Everything reliability-related is a method on the context:

- `ctx.run(...)` — durable step (retried, exactly-once)
- `ctx.get` / `set` / `clear` — durable keyed state
- `ctx.sleep(d)` / durable timers — survive restarts
- `ctx.call` / `send` — durable RPC (request-response, one-way, **delayed**)
- `ctx.awakeable()` / durable promises — wait for external events
- `TerminalException` — stop retries + drive Sagas

Plain Java in between. Reliability is a library call, not an architecture.

Part 3 — How does it work?

...and why it's super reliable

The core technique: journal + replay

For one invocation:

```
ctx.run("create user") —▶ result written to the log ✓ (committed)
ctx.run("subscribe paid") —▶ result written to the log ✓ (committed)
    ✨ CRASH here
```

recovery: replay the journal

- "create user" → already in log → SKIP (return saved result)
- "subscribe paid" → already in log → SKIP (return saved result)
- next step → not in log → EXECUTE for real

Completed steps are **replayed from the log**, not re-executed.

That's why side effects must go through `ctx`.

Where durability actually lives: the log

- Restate has its own **replicated log** (called **Bifrost**).
- A step is "done" the instant its entry is **replicated to a quorum** of nodes.
- The log is the **single source of truth** — everything else is rebuilt from it.

append step —▶ replicate to quorum —▶ ACK —▶ "it happened"

No external Kafka/Zookeeper. The replicated log is built in.

Partitions, leaders, and state

- Work is sharded into **partitions**; each has one **leader** processor (+ followers).
- The leader tails the log, invokes your handlers, and keeps a **materialized cache** of state in **RocksDB** for fast reads.
- That cache is **derivative** — it can always be rebuilt from the log. Not a second source of truth.
- Periodic **snapshots** → **S3** bound recovery time (replay only the tail).

Exactly-once, for real

- **Idempotency key** on a request → duplicates return the **same** committed result, no re-execution.

```
curl .../Account/acct-42/deposit -H 'idempotency-key: dep-001' --json '500'  
curl .../Account/acct-42/deposit -H 'idempotency-key: dep-001' --json '500'  
# balance moved by 500, not 1000
```

Why it's **super** reliable — recap

TL;DR — Restate handles the hard distributed-systems problems, so your code doesn't have to.

- Every step is saved the moment it finishes — a crash just **resumes**
- **Survives** node and server failures with no data loss
- **No duplicate** side effects, even under retries (exactly-once)
- **Automatic failover** — no manual recovery, no on-call scramble
- **One binary** — far less infrastructure to run (and to break)

Live demo

`SubscriptionService` · `Account` · `PaymentApprovalWorkflow`

see `demo/README.md` for the exact commands

Where it fits

- Payment / transfer flows that must be **exactly-once** and auditable
- Onboarding / servicing that **waits on humans or 3rd parties** for days
- Per-account state without standing up yet another datastore
- Replacing bespoke **retry + queue + cron + saga** plumbing with `ctx.*`
- Durable **AI agent** workflows

Try it in 10 minutes

```
brew install restatedev/tap/restate-server restatedev/tap/restate
restate-server                                # terminal 1
./gradlew run                                # terminal 2 (the demo)
restate deployments register http://localhost:9080
# UI at http://localhost:9070
```

- Docs: docs.restate.dev · Examples: github.com/restatedev/examples
- This deck + the Java demo: this repo

Questions & discussion